

Mini-RAG Technical Report

Mini-RAG: Recipe Knowledge Assistant

TEAM: 37

NAME : شدى أحمد سليمان
ID : 2023170291

NAME : سما محمد على
ID : 2023170270

NAME : سلمى فتحي سعيد
ID : 2023170261

NAME : فرح محمد مصطفى
ID : 2023170426

NAME : سلمى محمد حافظ
ID : 2023170265

NAME : هاجر إبراهيم شعبان
ID : 2023170683

1. Executive Summary

Mini-RAG is a containerized, end-to-end Retrieval-Augmented Generation (RAG) system built with FastAPI. The system solves the problem of intelligent natural-language querying over a corpus of HTML recipe documents, answering cooking questions (ingredients, steps, quantities) in both English and Arabic using only the content of the ingested files — never hallucinating from external knowledge.

The workflow is:

1. Ingest — HTML files are uploaded and stored under a project namespace.
2. Process — Files are parsed, cleaned, and split into text chunks stored in MongoDB.
3. Index — Chunks are embedded via a local sentence-transformer model and stored in Qdrant.
4. Query — User questions are embedded, matched against Qdrant via cosine similarity, and the top-k chunks are injected into a prompt sent to an OpenAI-compatible LLM endpoint.

3. API Documentation

- GET /api/
Purpose: Health / welcome check.
- POST /api/data/upload/{project_id}
Purpose: Upload an HTML file under the given project.
- POST /api/data/process/{project_id}
Purpose: Parse HTML content, split into chunks, and store in MongoDB.
- POST /api/nlp/index/push/{project_id}
Purpose: Generate embeddings and store them in Qdrant.
- POST /api/nlp/index/search/{project_id}
Purpose: Retrieve top-k similar chunks.
- POST /api/nlp/index/answer/{project_id}
Purpose: Full RAG pipeline with retrieval and answer generation.

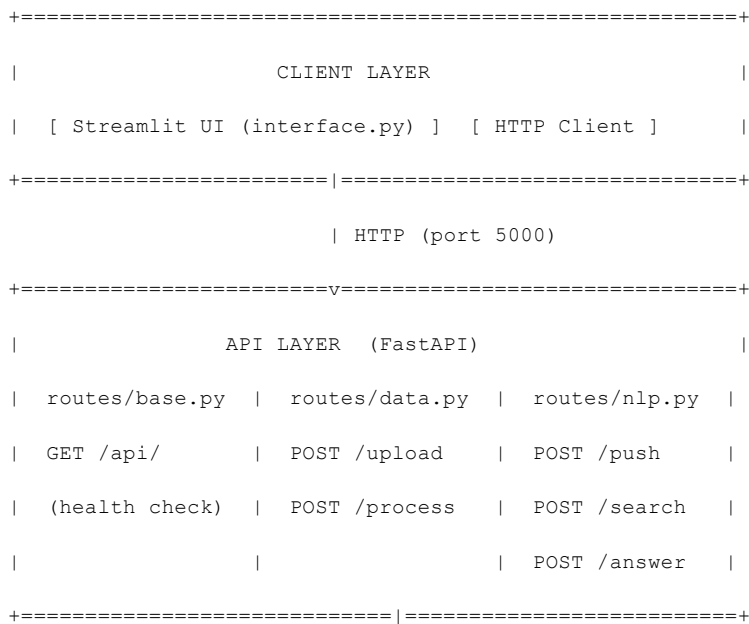
4. Embedding Model & Chunking Strategy

- ❖ Embedding Model:
 - sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
 - Embedding dimension: 384
 - Distance metric: Cosine similarity
- ❖ Chunking Strategy:
 - Chunk size: 500 tokens
 - Overlap: 50 tokens

5. Docker Deployment Instructions

- ❖ Prerequisites:
 - Docker Engine 24+
 - Docker Compose v2
- ❖ Build & Run:
 - docker compose up --build
- ❖ Containers:
 - MongoDB
 - Qdrant
 - FastAPI Application

System Architecture Diagram



```

+=====+
|          CONTROLLER LAYER   (Business Logic)          |
| DataController | ProcessController | NlpController |
| validate_file  | get_file_content | push_to_index |
| FileController | process_files    | search_vector |
| get_file_path  | (chunk+overlap)  | answer_rag    |
|      BaseController (parent of all controllers)      |
|  app_settings  | base_dir  | file_dir  | db_dir  |
+=====+-----+
|
|                                     v
|
+=====+
|          DATA LAYER   (MongoDB via Motor)          |
| ProjectModel (projects CRUD)                        |
| ChunkModel   (chunks CRUD, paginated async fetch)  |
| DataBaseModel (base: db_client + app_settings)      |
| MongoDB: [ projects collection ] [ chunks collection ]|
+=====+-----+
|
|                                     v
|
+=====+
|          STORES LAYER   (External Service Integrations) |
| LLM Store: LLMFactory --> OpenAIProvider            |
|                                     --> EmbeddingProvider |
|          LLMInterface (abstract contract)           |
| VectorDB Store: VectorDBFactory --> QdrantDB        |
|          VectorDBInterface (abstract contract)      |
| Template Store: TemplateParser --> locales/en/rag.py |
|                                     --> locales/ar/rag.py |
+=====+-----+
|
|                                     v
|
+=====+
|  INFRASTRUCTURE LAYER   (Docker / Compose)          |
| [ MongoDB container : port 27007 ]                  |
| [ Qdrant container   : port 6333 ]                  |
| docker-compose up --> entire stack runs            |
+=====+

```